Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

УЧЕБНЫЕ МАТЕРИАЛЫ СЕМИНАРА №4
СОЗДАНИЕ КЛАССОВ И КОЛЛЕКЦИЙ

Учебные материалы по дисциплине
«Архитектура автоматизированных систем обработки информации и управления»
Профиль: «Архитектура программных систем»

Москва – 2026

Оглавление

1 ЦЕЛЬ 3

2 ЧАСТЬ 1. СОЗДАНИЕ СИСТЕМЫ КЛАССОВ «ГЕОМЕТРИЧЕСКИЕ ФИГУРЫ» 3

2.1 Абстрактный класс «Геометрическая фигура» 3

2.2 Класс «Круг» 4

2.3 Класс «Прямоугольник» 4

2.4 Класс «Квадрат» 4

2.5 Фрагмент основной программы 5

3 ЧАСТЬ 2. СОЗДАНИЕ НЕСТАНДАРТНОЙ КОЛЛЕКЦИИ НА ОСНОВЕ СТАНДАРТНОЙ КОЛЛЕКЦИИ 7

3.1 Базовая реализация разреженной матрицы 7

3.2 Реализация механизма хранения ключа в словаре 13

3.2.1 Использование кортежа 13

3.2.2 Использование линейного индекса 14

3.2.3 Использование строковой конкатенации 14

3.2.4 Требования к ключам класса Dictionary 15

4 ЧАСТЬ 3. СОЗДАНИЕ НЕСТАНДАРТНОЙ КОЛЛЕКЦИИ БЕЗ ИСПОЛЬЗОВАНИЯ СТАНДАРТНЫХ КОЛЛЕКЦИЙ 16

4.1 Реализация простого списка 17

4.2 Реализация стека на основе списка 25

5 ВЫВОДЫ 27

1 Цель

Написать консольное приложение, которое реализует систему классов и позволяет создать собственные классы коллекций.

2 Часть 1. Создание системы классов «Геометрические фигуры»

2.1 Абстрактный класс «Геометрическая фигура»

```
/// <summary>
/// Абстрактная фигура
/// </summary>
/// <param name="type">Название типа фигуры</param>
internal abstract class Figure(string type)
{
    // Параметр "string type" задается при объявлении класса
    // Это механизм "первичного конструктора"

    /// <summary>
    /// Тип фигуры
    /// Только для чтения
    /// Инициализируется значением из первичного конструктора
    /// </summary>
    public string Type { get; } = type;

    /// <summary>
    /// Вычисление площади
    /// Вычисляемой свойство только для чтения
    /// </summary>
    public abstract double Area { get; }

    /// <summary>
    /// Приведение к строке, переопределение метода Object
    /// используются:
    /// - строковая интерполяция ($"...") для удобства
форматирования
    /// - expression-bodied member (=> вместо фигурных скобок)
для лаконичности
    /// </summary>
    public override string ToString() =>
                            $"{Type} площадью {Area}";
}
```

2.2 Класс «Круг»

Класс Круг наследуется от абстрактного класса Геометрическая фигура:

```
/// <summary>
/// Класс Круг
/// </summary>
/// <param name="radius"></param>
internal class Circle(double radius) : Figure("Круг")
{
    /// <summary>
    /// Площадь круга
    /// Переопределяем get-аксессор абстрактного свойства Area
    /// </summary>
    public override double Area => Math.PI * radius * radius;
}
```

2.3 Класс «Прямоугольник»

Класс Прямоугольник наследуется от абстрактного класса Геометрическая фигура:

```
/// <summary>
/// Класс Прямоугольник
/// </summary>
/// <param name="height">Высота</param>
/// <param name="width">Ширина</param>
/// <param name="type">Тип фигуры для удобства наследования</param>
internal class Rectangle(double height, double width, string type =
"Прямоугольник") : Figure(type)
{
    /// <summary>
    /// Площадь прямоугольника
    /// </summary>
    public override double Area => width * height;
}
```

2.4 Класс «Квадрат»

```
/// <summary>
/// Класс Квадрат
/// </summary>
/// <param name="size">размер стороны</param>
internal class Square(double size) : Rectangle(size, size, "Квадрат") {}
// никакой реализации для класа не требуется, специфика реализации
// задается передачей параметров в конструктор базового класса
```

2.5 Фрагмент основной программы

```
static void Main(string[] args)
{
    ////////////////////////////////
    #region Геометрические фигуры
    Rectangle rect = new(5, 4);
    Square square = new(5);
    Circle circle = new(5);

    Console.WriteLine(rect);
    Console.WriteLine(square);
    Console.WriteLine(circle);
    #endregion
}
```

Результат выполнения программы показан на рисунке 1.

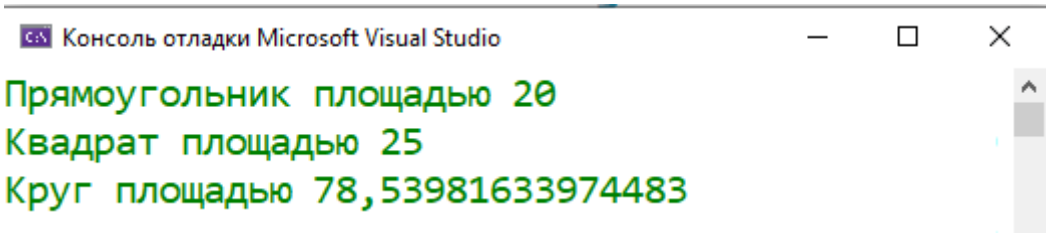

Рис. 1. Результат выполнения программы для геометрических фигур.

Блоки region можно сворачивать для удобства организации кода, пример показан на рисунках 2 и 3.

```
namespace seminar_4
{
    internal class Program
    {
        static void Main(string[] args)
        {
            ////////////////////////////////
            #region Геометрические фигуры
            Rectangle rect = new(5, 4);
            Square square = new(5);
            Circle circle = new(5);

            Console.WriteLine(rect);
            Console.WriteLine(square);
            Console.WriteLine(circle);
            #endregion
        }
    }
}
```

Рис. 2. Использование блока region.

```
namespace seminar_4
{
    internal class Program
    {
        static void Main(string[] args)
        {
            ////////////////////////////////
            Геометрические фигуры
        }
    }
}
```

Рис. 3. Использование блока region – свернутый блок.

3 Часть 2. Создание нестандартной коллекции на основе стандартной коллекции

3.1 Базовая реализация разреженной матрицы

В том случае, когда необходимая стандартная коллекция отсутствует в библиотеках .NET, прикладному программисту нужно самостоятельно реализовывать требуемую коллекцию. В этом случае возможны два подхода:

Использовать стандартную коллекцию и написать «надстройку» над ней для реализации требуемого поведения. Данный способ предпочтителен, так как требует относительно небольших ресурсов при кодировании.

Если предыдущий подход невозможен, то приходится создавать коллекцию полностью с нуля.

В данном разделе рассмотрен первый подход на примере класса «Разреженная матрица».

Разреженная матрица — тип данных, который достаточно часто используется в информатике. Это матрица большой размерности, которая может быть заполнена на 3–5%, остальные ячейки матрицы пусты. Использование в этом случае обычной матрицы нецелесообразно, так как потребует выделения очень больших объёмов памяти для хранения пустых значений.

Разреженную матрицу можно реализовать «поверх» стандартной коллекции. В приведённом ниже примере разреженная матрица реализована на основе словаря. Ключом элемента словаря является кортеж (tuple) из двух целых чисел — индексов ячейки матрицы по столбцу и строке, значением элемента словаря является значение элемента матрицы.

Перед собственно реализацией класса «Разреженная матрица» нам необходимо для тех элементов, которые будут помещаться в матрицу

задать поведение по умолчанию – необходимо сделать проверку на пустой элемент и возвращать этот пустой элемент. В нашем примере это реализовано с использованием интерфейса IMatrixCheckEmpty:

```
/// <summary>
/// Интерфейс для взаимодействия с пустыми элементами
/// </summary>
/// <typeparam name="T">Класс который предполагается помещать в ячейку
матрицы</typeparam>
public interface IMatrixCheckEmpty<T>
{
    /// <summary>
    /// Возвращает пустой элемент
    /// </summary>
    T GetEmptyElement();

    /// <summary>
    /// Проверка что элемент является пустым
    /// </summary>
    bool CheckEmptyElement(T element);
}
```

Интерфейс содержит два метода:

- GetEmptyElement, возвращающий пустой элемент разреженной матрицы;
- CheckEmptyElement, проверяющий, что переданное в качестве параметра метода значение является пустым элементом матрицы.

Методы данного интерфейса используются при создании разреженной матрицы.Создадим класс FigureMatrixCheckEmpty, который реализует проверку на пустоту для геометрической фигуры:

```
/// <summary>
/// Реализация проверки на пустоту для геометрической фигуры
/// </summary>
internal class FigureMatrixCheckEmpty : IMatrixCheckEmpty<Figure>
{
    /// <summary>
    /// В качестве пустого элемента возвращается null
    /// Обратите внимание на ! после null
    /// </summary>
    public Figure GetEmptyElement() => null!;

    /// <summary>
    /// Проверка что переданный параметр равен null
    /// </summary>
    public bool CheckEmptyElement(Figure element) => element is null;
```

```
}
```

Метод `Figure GetEmptyElement()` возвращает именно `Figure` а не `Figure?` то есть мы возвращает NULL, хотя тип не является Nullable. Это не является ошибкой, но в этом случае компилятор выдает предупреждение «Возможно, возврат ссылки, допускающей значение NULL». Если мы не хотим менять сигнатуру типа на Nullable и хотим сказать явно компилятору, что берем ответственность на себя, тогда используется null! (null-forgiving operator).

Рассмотрим реализацию класса «Разреженная матрица»:

```
/// <summary>
/// Класс разреженной матрицы
/// с первичным конструктором
/// </summary>
/// <typeparam name="T">Класс, объекты котрого предполагается помещать
в ячейку матрицы</typeparam>
/// <param name="maxX">Максимальное количество колонок матрицы</param>
/// <param name="maxY">Максимальное количество строк матрицы</param>
/// <param name="checkEmpty">Объект класса реализующего проверку на
пустоту</param>
public class Matrix<T>(int maxX, int maxY, IMatrixCheckEmpty<T>
checkEmpty)
{
    /// <summary>
    /// Словарь для хранения значений
    /// </summary>
    readonly Dictionary<(int x, int y), T> _matrix = [];

    /// <summary>
    /// Индексатор для доступа к данным
    /// </summary>
    public T this[int x, int y]
    {
        set
        {
            CheckBounds(x, y);
            _matrix[(x, y)] = value;
        }
        get
        {
            CheckBounds(x, y);
            return _matrix.TryGetValue((x, y), out var element)
                ? element
                : checkEmpty.GetEmptyElement();
        }
    }

    /// <summary>
    /// Проверка границ
```

```
    /// </summary>
    void CheckBounds(int x, int y)
    {
        if (x < 0 || x >= maxX)
            throw new ArgumentOutOfRangeException
                (nameof(x), $"x={x} выходит за границы");
        if (y < 0 || y >= maxY)
            throw new ArgumentOutOfRangeException
                (nameof(y), $"y={y} выходит за границы");
    }

    /// <summary>
    /// Ширина столбца при выводе матрицы (символов)
    /// </summary>
    public int ColumnWidth { get; set; } = 32;

    /// <summary>
    /// Приведение к строке в табличном виде
    /// </summary>
    public override string ToString()
    {
        var b = new StringBuilder();
        for (int j = 0; j < maxY; j++)
        {
            b.Append('|');
            for (int i = 0; i < maxX; i++)
            {
                string cell = !checkEmpty.CheckEmptyElement(this[i,
j])
                    ? $"{this[i, j]}"
                    : "-";
                b.Append(cell.PadRight(ColumnWidth));
                b.Append('|');
            }
            b.AppendLine();
        }
        return b.ToString();
    }
}
```

Основная структура данных для хранения разреженной матрицы — словарь _matrix. Ключ словаря — кортеж (int x, int y), содержащий индексы ячейки матрицы по столбцу и строке. Значение словаря — обобщённый тип T, который является типом значения элемента матрицы.

Размеры матрицы maxX, maxY и объект для работы с пустыми значениями checkEmpty передаются через первичный конструктор класса.

Передача checkEmpty через конструктор является разновидностью «внедрения зависимостей через конструктор».

Параметры первичного конструктора автоматически захватываются компилятором и доступны во всех методах класса — объявлять для них отдельные поля не требуется.

Обратите внимание на ключевое слово readonly в объявлении:

```
readonly Dictionary<(int x, int y), T> _matrix = [];
```

Ключевое слово readonly запрещает подменить сам словарь, но не запрещает изменять его содержимое. Пример кода:

```
void SomeMethod()
{
    // Случайно создали новый словарь вместо обращения к
существующему
    _matrix = [];  // Все данные потеряны! Без readonly —
компилятор разрешит такое присваивание
}
```

Наиболее важной частью реализации является индексатор «public T this[int x, int y]». В качестве параметров индексатору передаются координаты x и y — столбец и строка текущей ячейки матрицы.

Set-аксессор индексатора выполняет запись элемента в матрицу. Вначале выполняется проверка границ матрицы с помощью метода CheckBounds. Данный метод проверяет, что координаты текущей ячейки x и y находятся в диапазоне от 0 до границы по x или y соответственно. В случае выхода за границы генерируется исключение ArgumentOutOfRangeException с соответствующим сообщением. Если координаты текущей ячейки находятся в пределах требуемых границ, то выполняется запись значения value в словарь _matrix по ключу-кортежу (x, y). Запись выполняется через индексатор словаря, что позволяет при необходимости перезаписывать ранее установленное значение ячейки — аналогично поведению обычного массива. При необходимости запрета перезаписи можно добавить предварительную проверку наличия ключа в словаре.

Get-аксессор индексатора осуществляет чтение элемента из матрицы. Как и в случае записи, при чтении сначала проверяются границы матрицы

с помощью метода CheckBounds. Далее выполняется попытка извлечения значения из словаря с помощью метода TryGetValue. Данный метод за одно обращение к словарю проверяет наличие ключа и, если элемент найден, возвращает его через выходной параметр element. Если элемент не найден, возвращается пустое значение с использованием метода checkEmpty.GetEmptyElement().

В классе разреженной матрицы также переопределён метод ToString, выводящий значение матрицы в строковом представлении. Алгоритм работы данного метода является обычным, в нём с использованием двух вложенных циклов выполняется перебор и вывод в строку ячеек матрицы. Для работы со строками в данном методе используется класс StringBuilder. Зачем необходим этот класс, ведь можно создавать объекты класса String путём их конкатенации с помощью оператора «+»? Но класс String содержит неизменяемые строки, а применение оператора «+» приводит к порождению новых объектов, что плохо сказывается на производительности приложения. Когда класс String и оператор «+» используются в небольших фрагментах кода, то это не вызывает проблем. Но если этот способ активно применять в приложении, например для формирования больших текстовых отчётов, то будет создаваться большое количество ненужных строковых объектов, которые практически сразу должен будет удалить сборщик мусора. Разработчики платформы .NET предупреждают, что это может существенно снизить производительность приложения, поэтому в таком случае необходимо использовать класс StringBuilder, который позволяет формировать сложные строки без возникновения проблем с производительностью. У класса StringBuilder существует метод Append, который позволяет дописать в конец внутренней строки StringBuilder практически любые данные. Метод AppendLine дописывает данные с автоматическим добавлением переноса строки. Также для класса StringBuilder перегружен метод ToString,

который возвращает внутреннюю строку StringBuilder в виде объекта класса String.

Пример основной программы создания матрицы:

```
Console.WriteLine("\nМатрица");
Matrix<Figure> matrix = new(3, 3,
   new FigureMatrixCheckEmpty());
matrix[0, 0] = rect;
matrix[1, 1] = square;
matrix[2, 2] = circle;
Console.WriteLine(matrix);
```

```
Консоль отладки Microsoft Visual Studio
Прямоугольник площадью 20
Квадрат площадью 25
Круг площадью 78,53981633974483

Матрица
|Прямоугольник площадью 20      |-                            |-                            |
|-                              |Квадрат площадью 25          |-                            |
|-                              |-                            |Круг площадью 78,53981633974483 |
```

Рис. 4. Результат выполнения программы для геометрических фигур и разреженной матрицы.

3.2 Реализация механизма хранения ключа в словаре

Выбор ключа — важное проектное решение при реализации коллекции на основе словаря. Рассмотрим, какие варианты существуют для представления двумерных координат ячейки матрицы.

3.2.1 Использование кортежа

Кортеж (int x, int y) — вариант, использованный в рассмотренном примере:

```
Dictionary<(int x, int y), T> _matrix = [];

_matrix[(x, y)] = value;                                  // запись
_matrix.TryGetValue((x, y), out var element);              // чтение
```

Кортеж типов-значений (value tuple) — это структура, не требующая выделения памяти в куче (heap). Компилятор автоматически генерирует

для неё корректные методы GetHashCode и Equals, обеспечивая надёжную работу в качестве ключа словаря.

С точки зрения статьи https://matteding.github.io/2019/04/25/sparse-matrices/ в данном примере мы реализуем подход Dictionary Of Keys (DOK) к хранению разреженной матрицы.

3.2.2 Использование линейного индекса

Линейный индекс y * maxX + x — классический приём из линейной алгебры и компьютерной графики, отображающий двумерный индекс в одномерный:

```
Dictionary<int, T> _matrix = [];
int LinearIndex(int x, int y) => y * maxX + x;

_matrix[LinearIndex(x, y)] = value;                    // запись
_matrix.TryGetValue(LinearIndex(x, y), out var el);     // чтение
```

Для хранения ключа используется единственное целое число. Хеширование одного целого числа очень быстрая операция, поскольку хеш числа int равен самому числу. Также легко реализуется обратное преобразование: x = index % maxX, y = index / maxX, где в данном случае / это целая часть от деления с остатком, а % остаток от деления.

Недостаток метода в том, что схема хранения явно зависит от количества столбцов maxX. Если maxX будет изменяться динамически (в следующих версиях), то такая схема хранения не будет эффективной.

3.2.3 Использование строковой конкатенации

Строковая конкатенация x + "_" + y возможна при создании ключа в словаре:

```
Dictionary<string, T> _matrix = [];
string DictKey(int x, int y) => $"{x}_{y}";

_matrix[DictKey(x, y)] = value;  // запись
```

Достоинства подхода в том, что измерения матрицы явно разделимы, подход хорошо масштабируется в случае увеличения количества измерений.

Недостатки состоят в том, что при каждом обращении к словарю создаётся новый строковый объект в куче (что плохо с точки зрения производительности), хеширование строки выполняется побайтово, а разделитель _ (если его вдруг пропустить) критически важен для корректности — без него координаты (1, 11) и (11, 1) дали бы одинаковый ключ "111".

3.2.4 Требования к ключам класса Dictionary

Формально обобщённый параметр TKey в классе Dictionary<TKey, TValue> не имеет ограничений — компилятор примет любой тип. Однако корректная работа словаря гарантируется только при соблюдении ключом следующего контракта:

- Согласованность: если a.Equals(b) возвращает true, то a.GetHashCode() и b.GetHashCode() обязаны возвращать одинаковое значение.
- Стабильность: пока объект является ключом словаря, результат его GetHashCode() не должен меняться.

Типы-значения (int, string, double), перечисления, кортежи типов-значений и record struct соблюдают этот контракт автоматически — они сравниваются по значению.

Обычные классы (class) по умолчанию сравниваются по ссылке, что может привести к неожиданному поведению:

```
class Point
{
        public int X { get; set; }
        public int Y { get; set; }
}

var dict = new Dictionary<Point, string>();
dict[new Point { X = 1, Y = 2 }] = "A";
```

```
dict.ContainsKey(new Point { X = 1, Y = 2 }); // false! Другой объект
```

Два объекта Point с одинаковыми координатами — это разные ссылки, поэтому словарь не находит элемент. Для корректной работы необходимо переопределить методы GetHashCode и Equals.

Ещё более опасная ситуация — мутация ключа после добавления в словарь:

```
var p = new Point { X = 1, Y = 2 };
dict[p] = "A";
Console.WriteLine(dict[p]); // "A"

p.X = 99; // Мутация ключа!
Console.WriteLine(dict[p]); // KeyNotFoundException
```

Хеш объекта изменился, элемент «потерян» навсегда. Он существует в памяти, но извлечь его невозможно, и никакого предупреждения при этом не выдаётся.

После плохого примера с классом рассмотрим «хороший» пример с record struct:

```
record struct Point(int X, int Y);

var dict = new Dictionary<Point, string>();
dict[new Point(1, 2)] = "A";
dict.ContainsKey(new Point(1, 2)); // true!
```

Пример работает, потому что record struct сравнивается по значению. Для record struct компилятор автоматически генерирует GetHashCode и Equals на основе всех полей.

4 Часть 3. Создание нестандартной коллекции без использования стандартных коллекций

В данном разделе разбирается пример создания коллекции в том случае, когда использовать стандартные коллекции нецелесообразно.

Например, это реализация нестандартных деревьев, что достаточно трудно сделать на основе стандартных коллекций.

Однако сама по себе реализация дерева — сложная задача. Поэтому здесь в качестве учебного примера создаётся простой однонаправленный список и в качестве его наследника реализуется класс стека. Рассматриваемые классы создаются полностью с нуля без использования стандартных коллекций.

4.1 Реализация простого списка

Для элемента списка можно было бы реализовать отдельный интерфейс или использовать для списков обобщенные типы, мы будем использовать обобщенные типы.

Класс SimpleListItem реализует контейнерный элемента списка:

```
/// <summary>
/// Контейнерный элемент списка
/// </summary>
/// <typeparam name="T">Тип элемента списка</typeparam>
/// <param name="data">Параметр, соответствующий типу элемента
списка</param>
public class SimpleListItem<T>(T data)
{
    /// <summary>
    /// Данные
    /// </summary>
    public T Data { get; set; } = data;

    /// <summary>
    /// Следующий элемент
    /// </summary>
    public SimpleListItem<T>? Next { get; set; }
}
```

Тип-обобщение T — тип данных списка. Класс SimpleListItem содержит свойство Data обобщённого типа, предназначенное для хранения данных.

В классе SimpleListItem также содержится свойство Next типа SimpleListItem<T>?, являющееся аналогом указателя на следующий

элемент. Знак ? после типа означает, что свойство может содержать null — это естественно, так как у последнего элемента списка нет следующего элемента и тогда применяется значение NULL. Данное объявление может показаться несколько странным по сравнению с языком C++, где при создании собственных структур данных используются указатели. Здесь свойство Next кажется скорее вложенным в предыдущий элемент, так что получается своеобразная матрёшка из вложенных элементов списка. Однако синтаксис в этом случае не совсем точно отражает работу с памятью. Классы в .NET являются ссылочными типами, следовательно, при объявлении переменной типа объект класса в данной переменной данные будут храниться именно как ссылка на объект класса, то есть фактически как указатель. Поэтому в памяти последовательность объектов класса SimpleListItem будет представлена в виде цепочки ссылок.

Класс SimpleList реализует список:

```
/// <summary>
/// Список
/// </summary>
public class SimpleList<T> : IEnumerable<T>
    where T : IComparable
{
    /// <summary>
    /// Первый элемент списка
    /// </summary>
    protected SimpleListItem<T>? first;
    //В C# поля ссылочных типов всегда
    //инициализируются значением null по умолчанию
    //Следующее объявление корректно но избыточно:
    //protected SimpleListItem<T>? first = null;

    /// <summary>
    /// Последний элемент списка
    /// </summary>
    protected SimpleListItem<T>? last;

    /// <summary>
    /// Количество элементов
    /// </summary>
    public int Count { get; protected set; }

    /// <summary>
```

```
/// Добавление элемента
/// </summary>
public void Add(T element)
{
    var newItem = new SimpleListItem<T>(element);
    Count++;

    // Добавление первого элемента
    if (last is null)
    //if (first is null) - также корректно
    //но тогда для last.Next = newItem;
    //будет выдаваться предупреждение:
    //"Разыменование вероятной пустой ссылки."
    {
        first = newItem;
        last = newItem;
    }
    // Добавление следующих элементов
    else
    {
        last.Next = newItem;
        last = newItem;
    }
}

/// <summary>
/// Чтение контейнера с заданным номером
/// </summary>
public SimpleListItem<T> GetItem(int number)
{
    if (number < 0 || number >= Count)
        throw new IndexOutOfRangeException(
            $"Индекс {number} выходит за границы списка");

    var current = first;
    for (int i = 0; i < number; i++)
        current = current!.Next;

    return current!;
}

/// <summary>
/// Чтение элемента с заданным номером
/// </summary>
public T Get(int number) => GetItem(number).Data;

/// <summary>
/// Для перебора коллекции
/// </summary>
public IEnumerator<T> GetEnumerator()
```

```
    {
        var current = first;
        while (current is not null)
        {
            yield return current.Data;
            current = current.Next;
        }
    }

    // Реализация обобщённого IEnumerator<T> требует реализации
    // необобщённого интерфейса IEnumerable
    System.Collections.IEnumerator
System.Collections.IEnumerable.GetEnumerator()
        => GetEnumerator();

    /// <summary>
    /// Сортировка
    /// </summary>
    public void Sort() => Sort(0, Count - 1);

    /// <summary>
    /// Алгоритм быстрой сортировки
    /// </summary>
    private void Sort(int low, int high)
    {
        int i = low;
        int j = high;
        T x = Get((low + high) / 2);
        do
        {
            while (Get(i).CompareTo(x) < 0) ++i;
            while (Get(j).CompareTo(x) > 0) --j;
            if (i <= j)
            {
                Swap(i, j);
                i++; j--;
            }
        } while (i <= j);

        if (low < j) Sort(low, j);
        if (i < high) Sort(i, high);
    }

    /// <summary>
    /// Вспомогательный метод для обмена элементов при
сортировке
    /// </summary>
    private void Swap(int i, int j)
    {
        var ci = GetItem(i);
```

```
        var cj = GetItem(j);
        (ci.Data, cj.Data) = (cj.Data, ci.Data);
    }
}
```

В методе GetItem используется null-forgiving operator. Компилятор видит, что current имеет тип SimpleListItem<T>? и может быть null. Но программист знает, что после проверки границ number < Count переменная current гарантированно не равна null на каждой итерации цикла. Оператор ! сообщает компилятору: «здесь null невозможен, я это проверил».

Рассмотрим сначала поля и методы, отвечающие за внутреннее устройство списка.

Поля first и last содержат контейнеры (вернее, ссылки на контейнеры) для первого и последнего элемента списка. Тип полей — SimpleListItem<T>? с вопросительным знаком, что явно указывает на возможность значения null (например, у пустого списка оба поля равны null).

Свойство Count содержит количество элементов. Для чтения оно доступно везде, в том числе и снаружи класса (область видимости public), а для записи — только в классе и его наследниках (область видимости protected).

Метод добавления нового элемента в конец списка Add создаёт новый контейнер элемента на основе переданных данных (объект класса SimpleListItem), увеличивает количество элементов списка на единицу, а затем добавляет контейнер к цепочке контейнеров. В результате этого поле first будет содержать ссылку на первый, а last — на последний элементы списка.

Метод GetItem используется для получения контейнера по его порядковому номеру, в частности для контейнера с номером N метод перебирает в цикле N−1 контейнеров и возвращает N-ый контейнер. Метод Get возвращает данные, находящиеся в контейнере.

Рассмотренные свойства и методы являются основными для работы списка. Кроме этого, для класса списка реализована дополнительная функциональность.

Данные списка можно перебирать в цикле foreach. Чтобы в .NET элементы коллекции можно было перебирать в цикле foreach, класс коллекции должен реализовывать интерфейс IEnumerable. Класс SimpleList<T> реализует обобщённый интерфейс IEnumerable<T>. Реализация данного интерфейса требует реализации у класса методов GetEnumerator.

Обобщённый метод public IEnumerator<T> GetEnumerator() осуществляет перебор всех элементов списка и их возврат с помощью конструкции yield return. Реализация обобщённого интерфейса IEnumerator<T> требует реализации необобщённого интерфейса IEnumerator. Поэтому класс SimpleList также реализует необобщённый метод System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator(), который вызывает обобщённую реализацию GetEnumerator.

Также для простого списка реализована сортировка методом быстрой сортировки (Quick Sort). Метод Sort без параметров выполняет сортировку, метод Sort с параметрами применяется для внутренних рекурсивных вызовов. Вспомогательный метод Swap используется для перестановки элементов. Обмен значений в методе Swap выполняется с помощью деконструкции кортежа — стандартного приёма в современном C#, позволяющего обменять значения двух переменных одной строкой без промежуточной переменной:

```
(ci.Data, cj.Data) = (cj.Data, ci.Data);
```

Алгоритм быстрой сортировки разработал Чарльз Хоар во время стажировки в МГУ имени М.В. Ломоносова в 1960 году. Интересно то, что алгоритм изначально создавался для сортировки данных на магнитной ленте (устройстве с последовательным доступом), однако в результате он

оказался одним из самых эффективных и для сортировки коллекций с произвольным доступом к элементам.

Идея алгоритма состоит в том, что выбирается некоторый средний элемент коллекции и предполагается, что он находится на своём месте. Далее перебираются все элементы коллекции слева и справа от него. Если слева расположен больший элемент, а справа меньший, то они меняются местами. В результате такого прохода выбранный элемент действительно оказывается на своём месте, слева от него находятся все меньшие, а справа все большие элементы, однако элементы справа и слева не упорядочены. Поэтому далее алгоритм вызывается отдельно для левой и для правой части, где также выбираются средние элементы и работа алгоритма рекурсивно повторяется до тех пор, пока размер сортируемого подмассива не будет равен одному элементу.

Несмотря на кажущуюся простоту алгоритма, доказано, что он оказался одним из самых эффективных алгоритмов сортировки. Одной из проблем алгоритма является большое количество рекурсивных вызовов, что может переполнить стек вызовов. Поэтому в некоторых реализациях данного алгоритма рекурсию заменяют на цикл, а вместо стека вызовов используют отдельный стек данных, где хранят индексы начала и конца сортируемого подмассива.

В данный момент при попытке скомпилировать код будет выдаваться ошибка «Тип "seminar_4.Figure" не может быть использован как параметр типа "T" в универсальном типе или методе "SimpleList<T>". Нет преобразования неявной ссылки из "seminar_4.Figure" в "System.IComparable"», потому что для класса Figure не реализован интерфейс IComparable, позволяющий сравнивать два элемента что является основной для сортировки данных. Добавим в класс Figure реализацию для сравнения элементов на основе площади, наследуемые

классы унаследуют эту реализацию от базового класса и проект будет компилироваться корректно:

```
/// <summary>
/// Абстрактная фигура
/// </summary>
/// <param name="type">Название типа фигуры</param>
internal abstract class Figure(string type) : IComparable
{
    // Параметр "string type" задается при объявлении класса
    // Это механизм "первичного конструктора"

    /// <summary>
    /// Тип фигуры
    /// Только для чтения
    /// Инициализируется значением из первичного конструктора
    /// </summary>
    public string Type { get; } = type;

    /// <summary>
    /// Вычисление площади
    /// Вычисляемой свойство только для чтения
    /// </summary>
    public abstract double Area { get; }

    /// <summary>
    /// Приведение к строке, переопределение метода Object
    /// используются:
    /// - строковая интерполяция ($"...") для удобства
форматирования
    /// - expression-bodied member (=> вместо фигурных скобок)
для лаконичности
    /// </summary>
    public override string ToString() =>
                    $"{Type} площадью {Area}";

    /// <summary>
    /// Сравнение фигур по площади (по возрастанию)
    /// </summary>
    public int CompareTo(object? obj) =>
        obj is Figure other
            ? Area.CompareTo(other.Area)
            : throw new ArgumentException("Объект не является
фигурой");
}
```

Рассмотрим пример использования класса SimpleList:

```
//*******************************************************
// Список
//*******************************************************
```

```
SimpleList<Figure> list = new();
list.Add(circle);
list.Add(rect);
list.Add(square);

Console.WriteLine("\nПеред сортировкой списка:");
foreach (var x in list) Console.WriteLine(x);

list.Sort();

Console.WriteLine("\nПосле сортировки списка:");
foreach (var x in list) Console.WriteLine(x);
```

И результат выполнения программы:

```
Консоль отладки Microsoft Visual Studio
Прямоугольник площадью 20
Квадрат площадью 25
Круг площадью 78,53981633974483

Матрица
|Прямоугольник площадью 20      |-                              |-                              |
|-                              |Квадрат площадью 25            |-                              |
|-                              |-                              |Круг площадью 78,53981633974483 |


Перед сортировкой списка:
Круг площадью 78,53981633974483
Прямоугольник площадью 20
Квадрат площадью 25

После сортировки списка:
Прямоугольник площадью 20
Квадрат площадью 25
Круг площадью 78,53981633974483
```

Рис. 5. Результат выполнения программы для геометрических фигур, разреженной матрицы и списка.

4.2 Реализация стека на основе списка

Класс стека наследуется от класса списка:

```
/// <summary>
/// Класс стек
/// </summary>
class SimpleStack<T> : SimpleList<T> where T : IComparable
{
    /// <summary>
    /// Добавление в стек
    /// </summary>
    public void Push(T element) => Add(element);

    /// <summary>
    /// Удаление и чтение из стека
```

```
    /// </summary>
    public T Pop()
    {
        T result;

        // Если стек пуст, возвращается значение по умолчанию
для типа
        if (Count == 0)
            return default!;
        else if (Count == 1)
        {
            // Единственный элемент — читаем и обнуляем список
            result = first!.Data;
            first = null;
            last = null;
        }
        else
        {
            // Поиск предпоследнего элемента
            var newLast = GetItem(Count - 2);
            // Чтение значения из последнего элемента
            result = newLast.Next!.Data;
            // Предпоследний элемент становится последним
            last = newLast;
            newLast.Next = null;
        }

        Count--;
        return result;
    }
}
```

Рассмотрим пример использования класса стека:

```
//*********************************************************
// Стек
//*********************************************************
SimpleStack<Figure> stack = new();

// Добавление данных в стек
stack.Push(rect);
stack.Push(square);
stack.Push(circle);

Console.WriteLine("\nВывод данных стека:");
// Чтение данных из стека
while (stack.Count > 0)
{
    Figure f = stack.Pop();
    Console.WriteLine(f);
}
```

И результат выполнения программы:

```
Выбрать Консоль отладки Microsoft Visual Studio
Прямоугольник площадью 20
Квадрат площадью 25
Круг площадью 78,53981633974483

Матрица
|Прямоугольник площадью 20       |-                                |-                                |
|-                               |Квадрат площадью 25              |-                                |
|-                               |-                                |Круг площадью 78,53981633974483 |


Перед сортировкой списка:
Круг площадью 78,53981633974483
Прямоугольник площадью 20
Квадрат площадью 25

После сортировки списка:
Прямоугольник площадью 20
Квадрат площадью 25
Круг площадью 78,53981633974483

Вывод данных стека:
Круг площадью 78,53981633974483
Квадрат площадью 25
Прямоугольник площадью 20
```

Рис. 6. Результат выполнения программы для геометрических фигур, разреженной матрицы, списка и стека.

5 Выводы

В данном семинаре мы реализовали собственную систему классов геометрических фигур, а также создали собственные классы коллекций.